

Making LLMs more Specialized & Practical

Fine Tuning, Pruning, Quantizing, Distilling

Dr. Sreerama K. Murthy

22nd April 2026

Yesterday ...

1. What is the physical architecture of a **Large Language Model**?
2. How is text fed into it? (**embedding**, encoding)
3. What training prepares a transformer into a **foundation model**?

Today...

Except as public intelligence infrastructure, LLMs are too general-purpose and too big – hence guzzling too much data, compute, energy, funds, etc. Hence..

1. How to convert foundation models into specialists? (Fine tuning)
 - How to fine tune efficiently?
2. What techniques are available to shrink LLMs into Small LMs, Mini LMs, Micro LMs, Nano LMs?
 - Pruning
 - Quantization
 - Distillation

Fine-Tuning & PEFT

When prompts aren't enough... teach the model new behavior

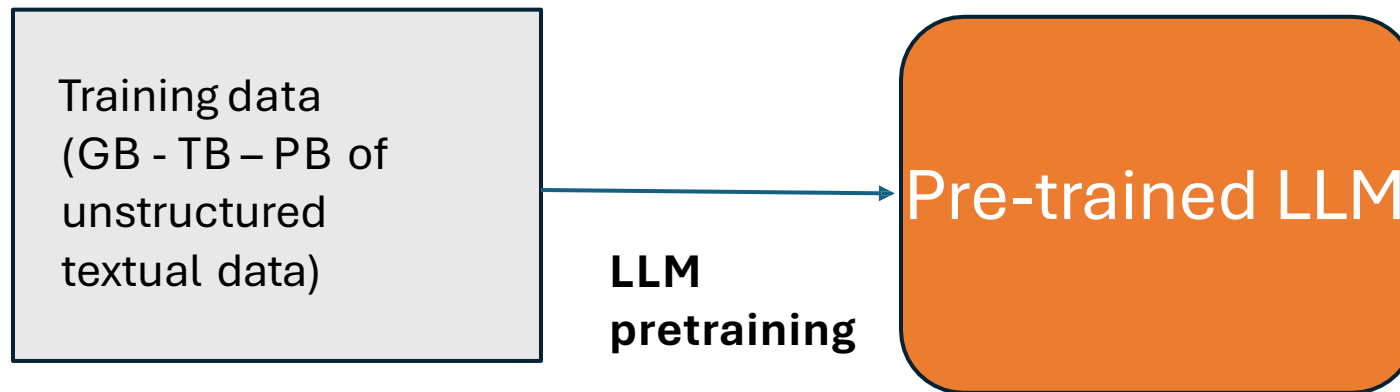
What Is The Need For Finetuning?

- In-context learning may not be able to include enough data samples to describe the problem well.
- In-context learning may not work for smaller LLMs
- Examples take up space in the context window.
- While LLM may be good with natural language understanding and generation, task can be specific.
 - Conversations; Code generation; Summarization; Sentiment analysis
- Finetuned small language models (SLMs) can perform very well.

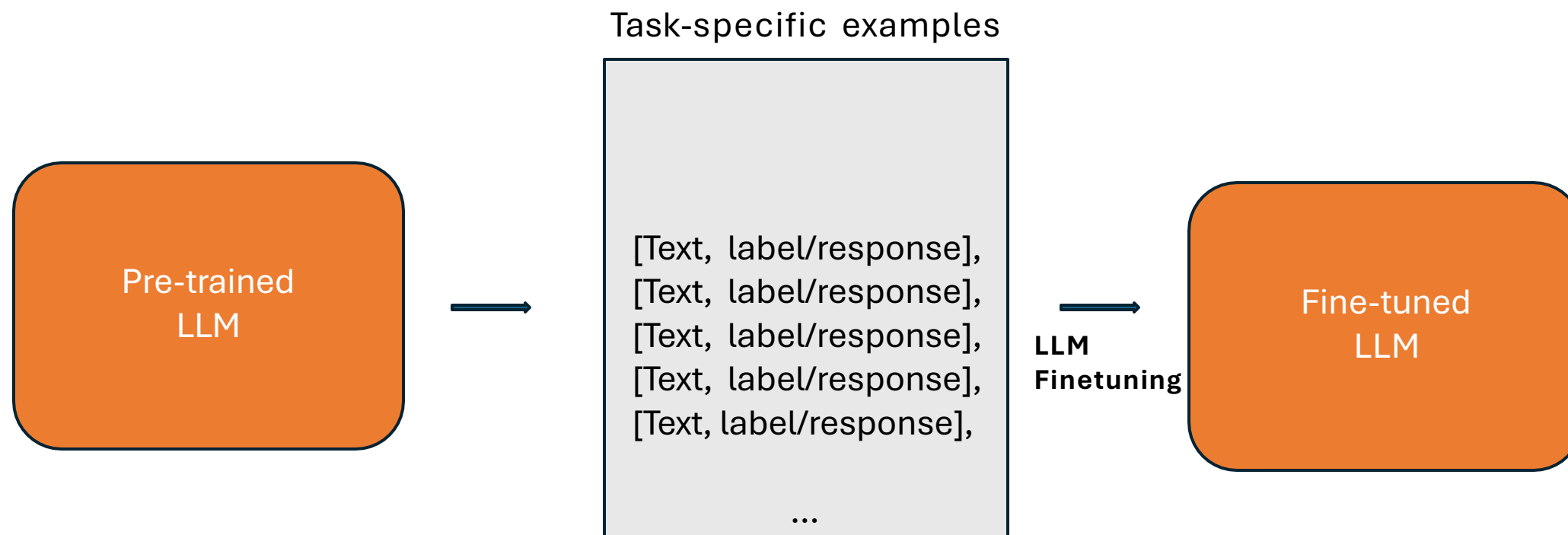
Why Fine-tune An LLM?

- Prompting = surface-level control
- Fine-tuning = embed knowledge or style into the model
- Prompting (temporary fix) vs Fine-Tuning (permanent learning)
- Start with prompting.
*Use RAG if the model needs **external knowledge**.*
Use fine-tuning when the model itself needs to change how it behaves.

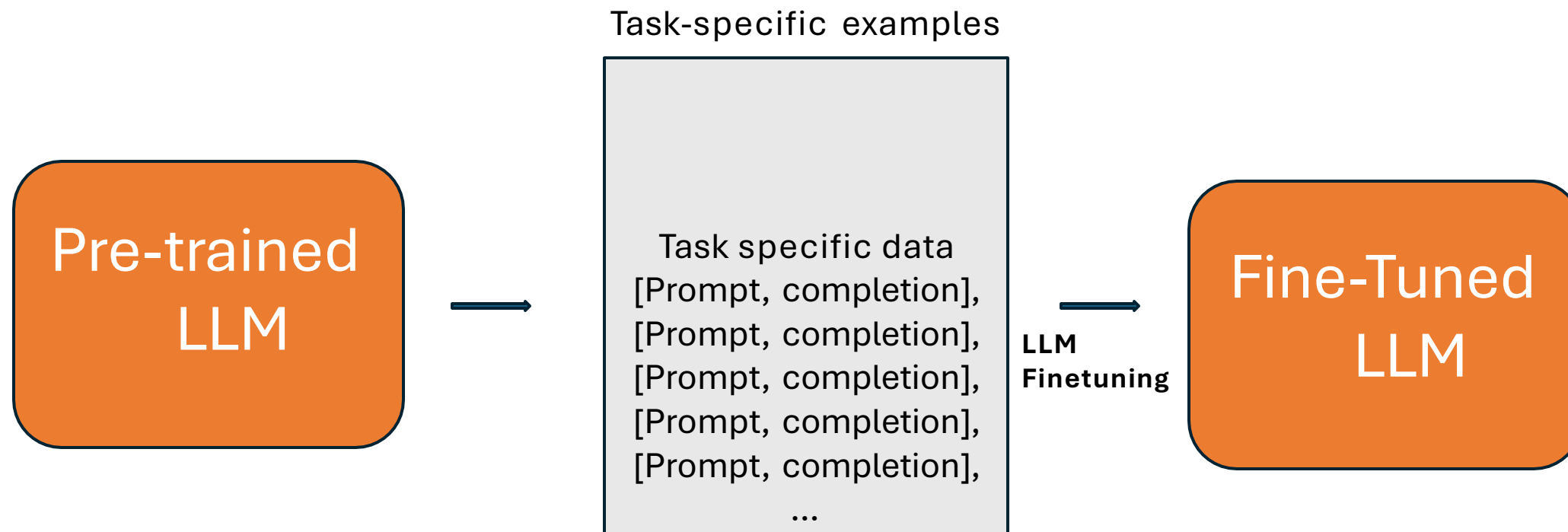
LLM finetuning (high-level)



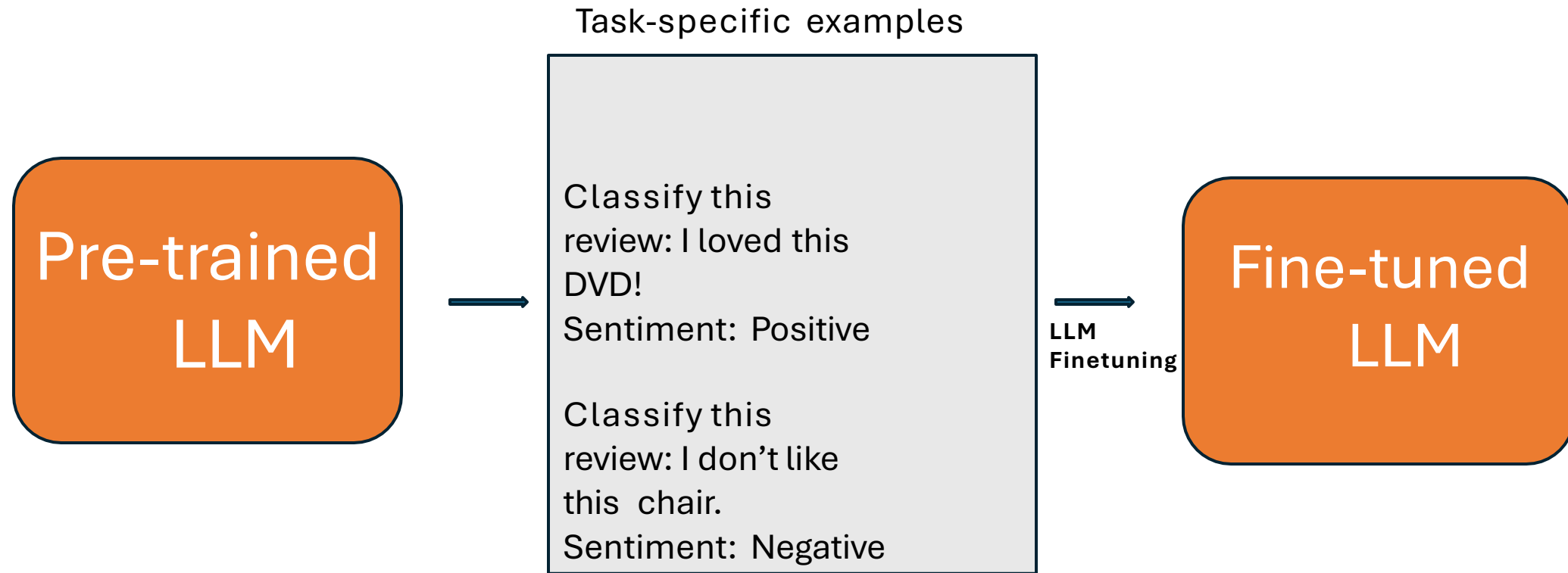
LLM Finetuning (contd.)



Using Prompts To Fine-tune LLMs



Using prompts to fine-tune LLMs



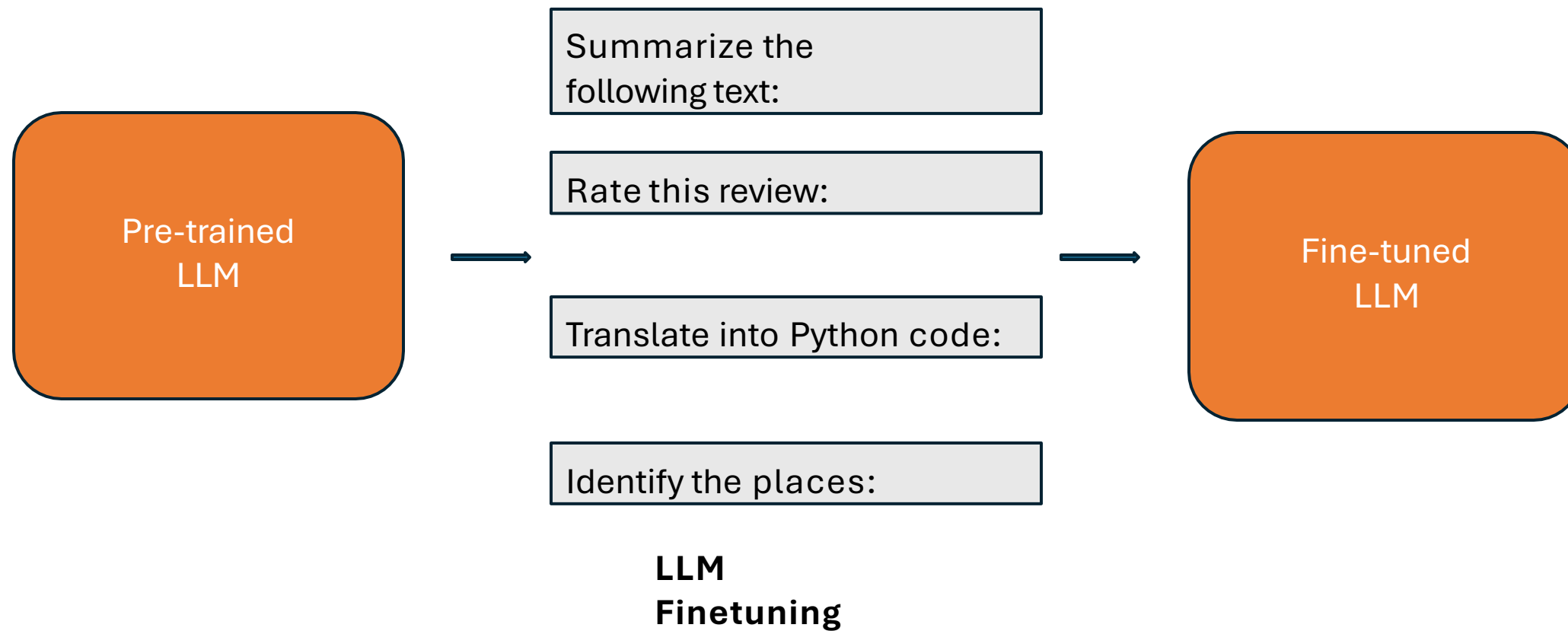
Effect Of Tuning On A Single Task

Catastrophic forgetting

- Only few thousand samples needed for finetuning on a task.
- Learning on a single task with less data can lead to reduction in ability on other tasks



Multi-task finetuning of LLM

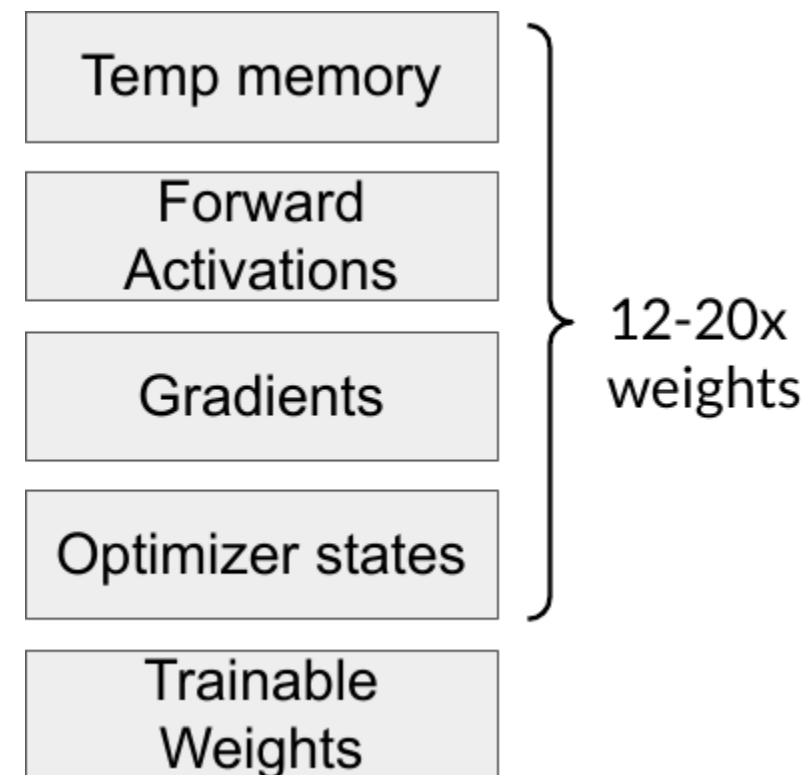


PEFT

Parameter Efficient Fine Tuning

Full fine-tuning of large LLMs is challenging

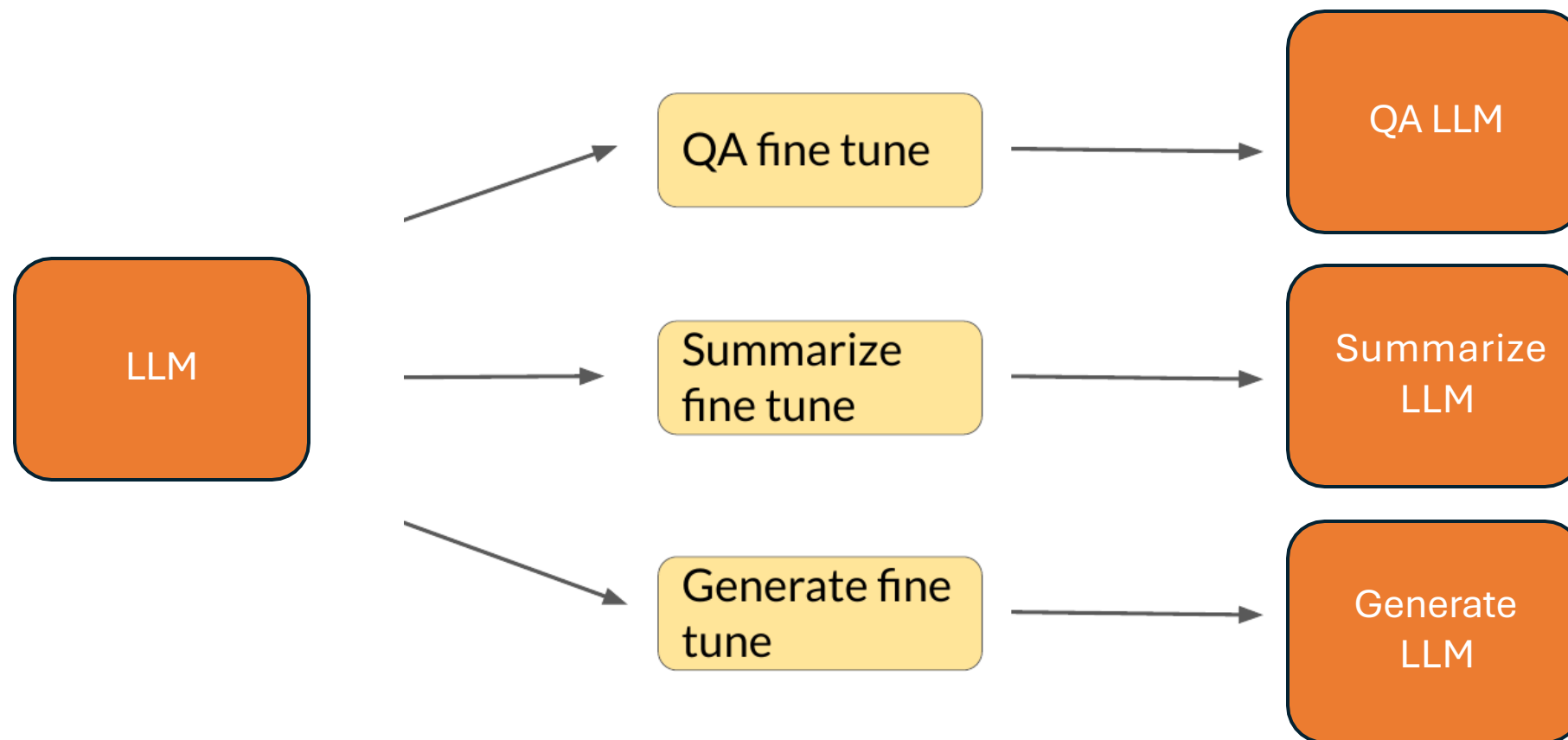
- Full finetuning requires large number of computational resources.
- GPU memory requirement is equivalent to training the LLM from scratch.
- High budget implications.
- Need to create a large finetuned LLM per task or a group of tasks.



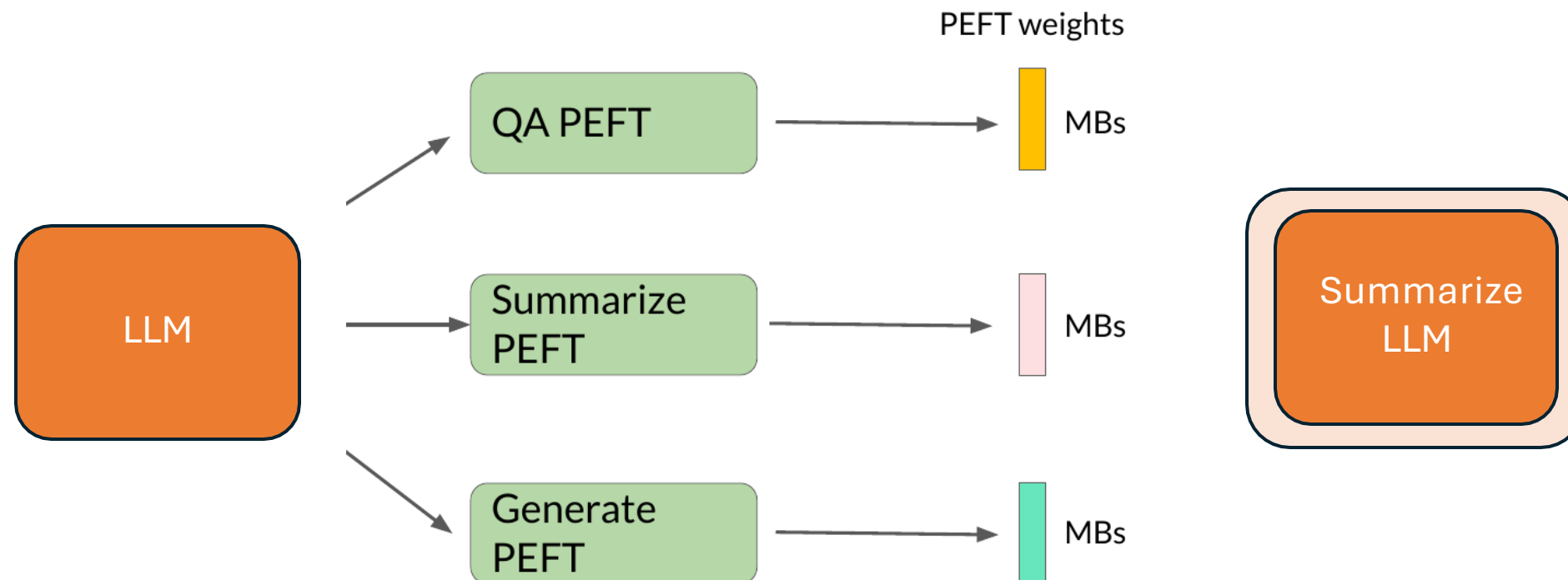
Parameter Efficient Fine-tuning (PEFT)

- ✓ Only train a small set of additional parameters of the LLM
- ✓ Most layers of the LLM are frozen
- ✓ Less prone to catastrophic forgetting
- ✓ Less storage requirement
- ✓ Computationally less expensive
- ✓ Less budget requirement

Finetuning creates a full copy of the LLM



PEFT finetuning saves cost and storage



PEFT Methods

Reparameterization

Reparameterize model weights using a low-rank representation

LoRA

Additive

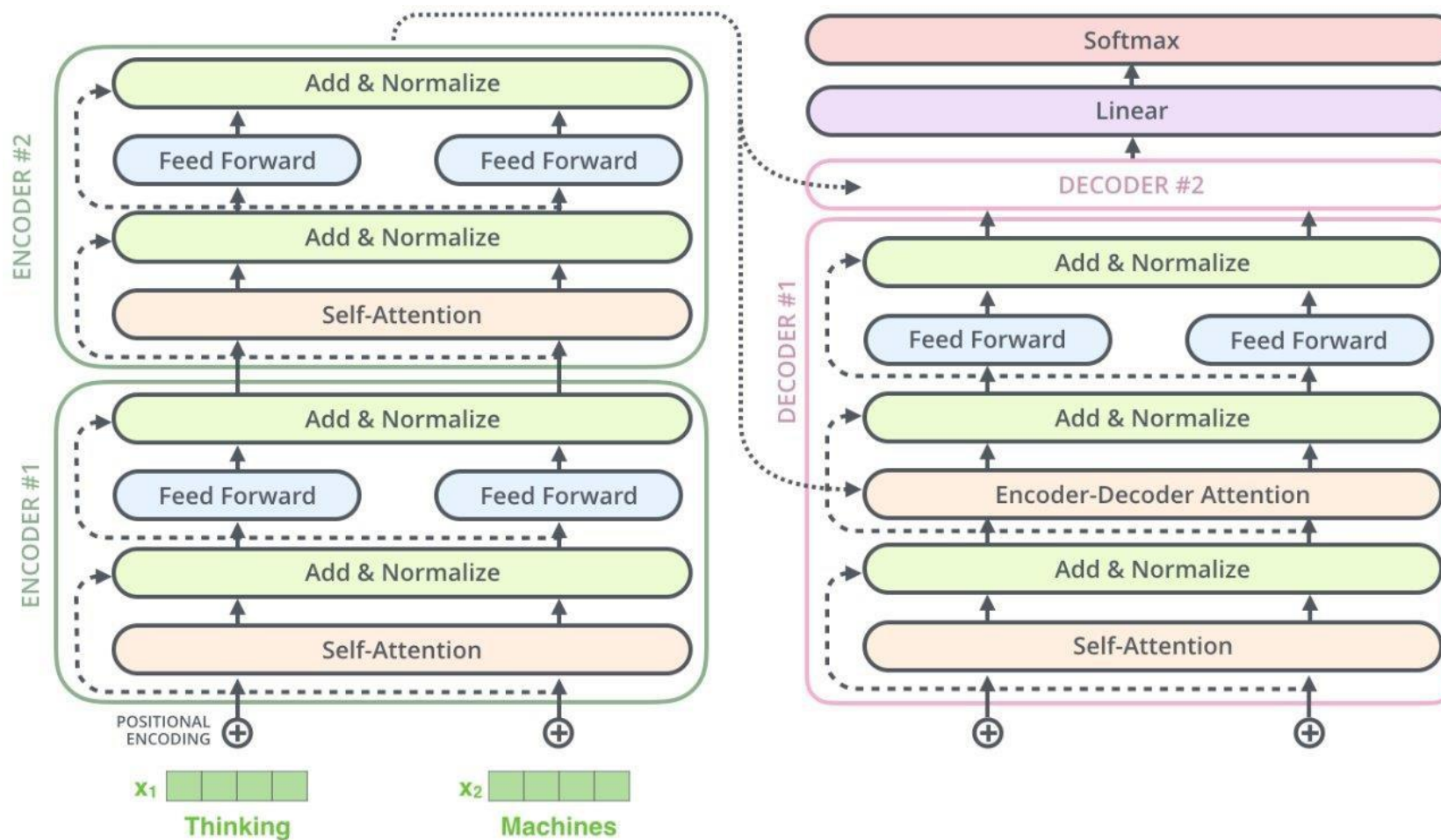
Add trainable layers or parameters to model

Adapters

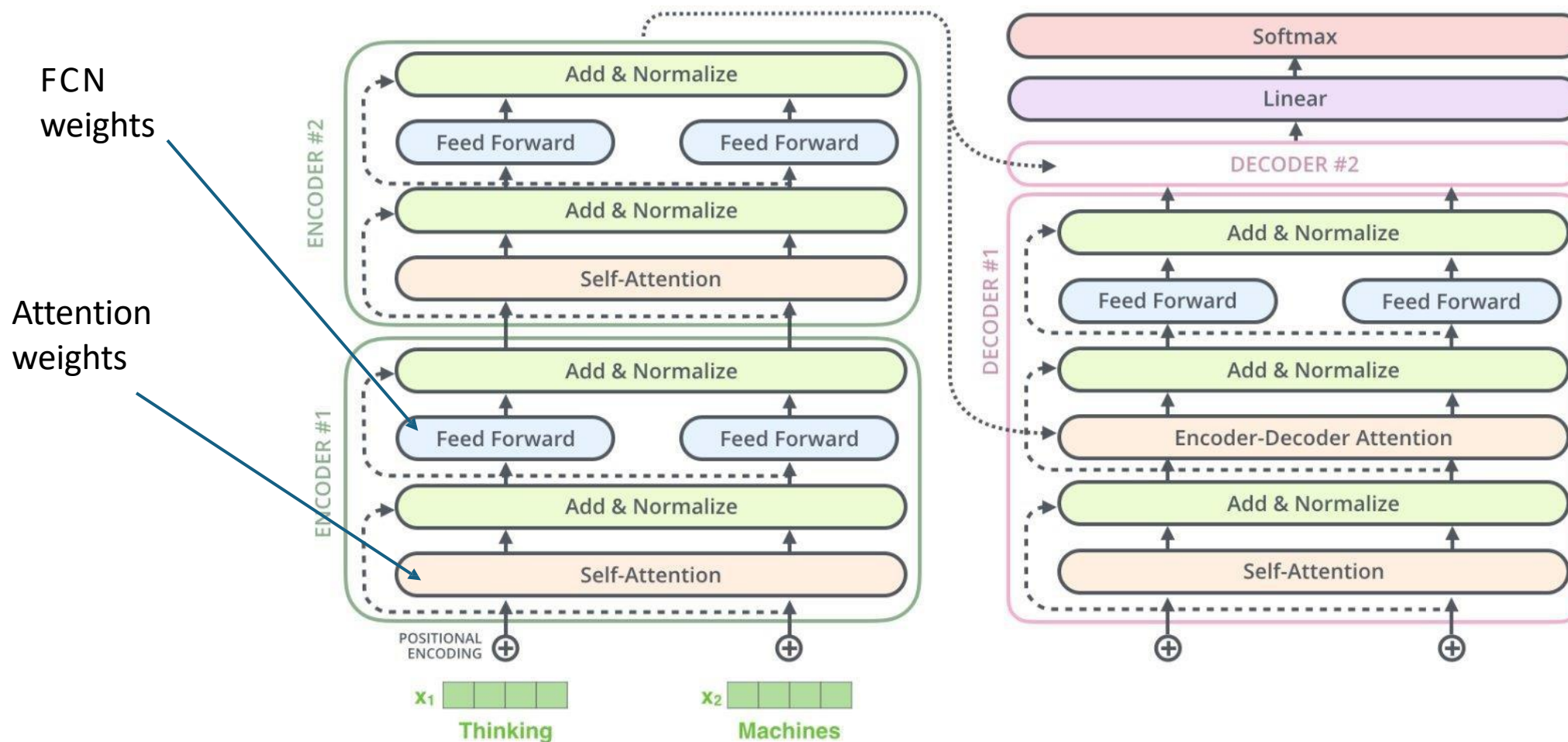
Soft Prompts

Prompt Tuning

Transformers Recap

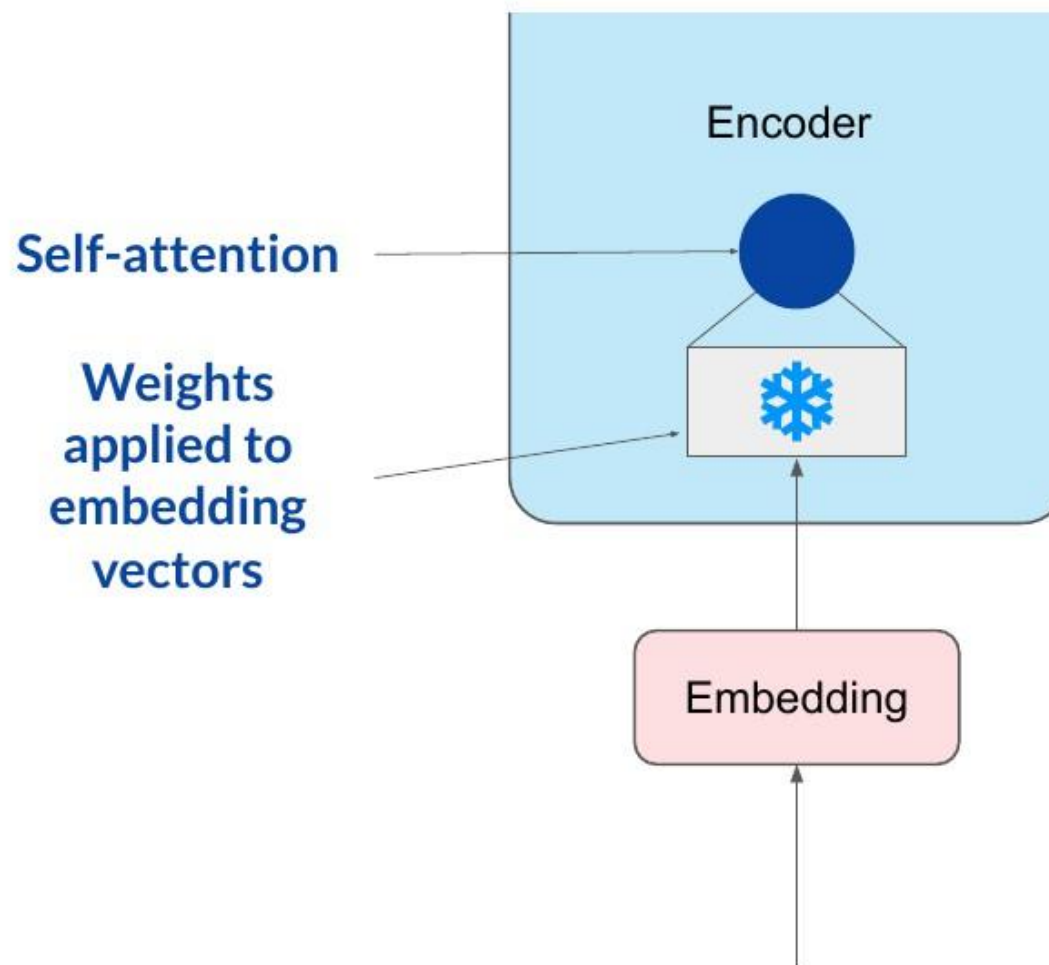


Transformers Recap



LoRA: Low Rank Adaption of LLMs

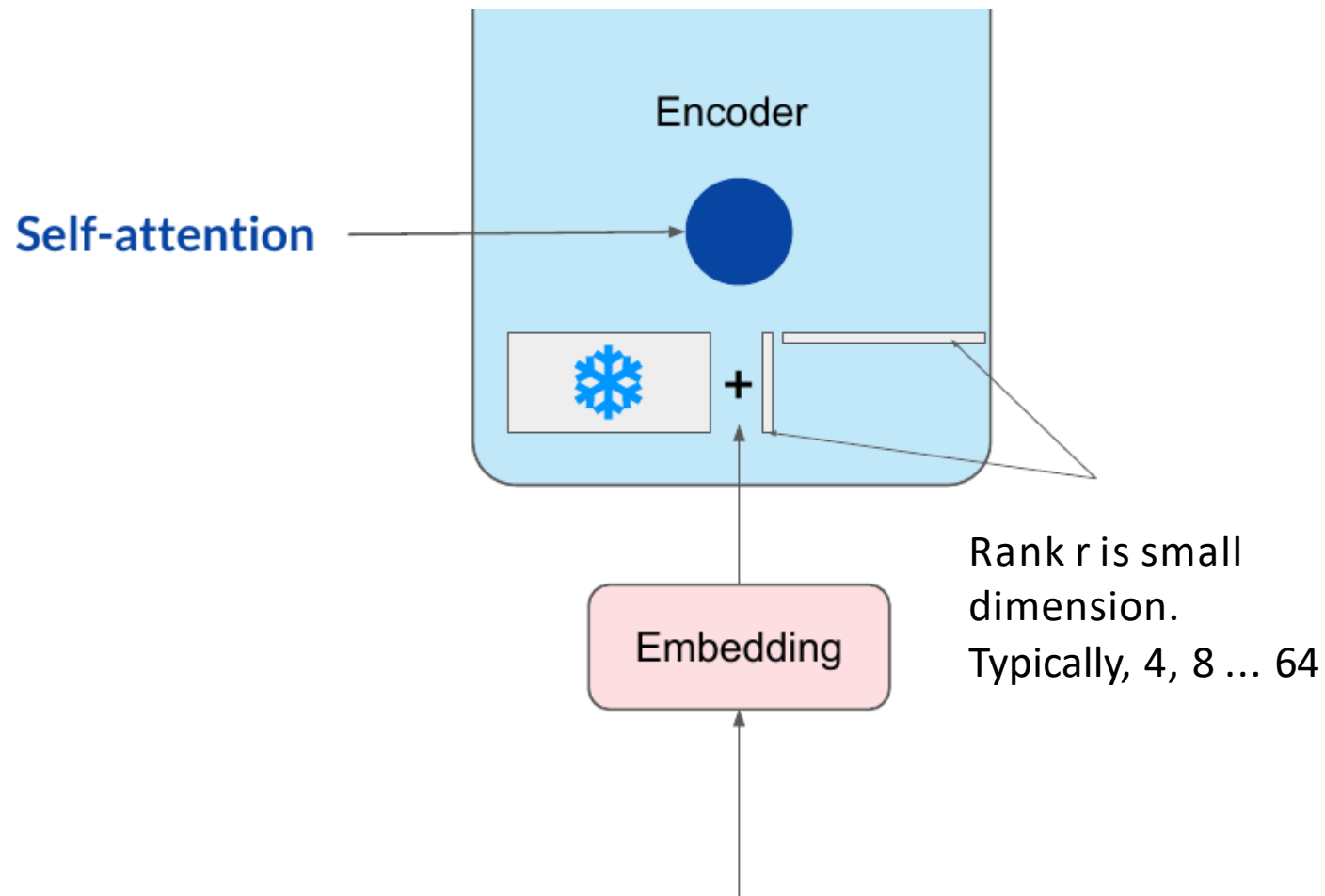
Step-1: Freeze most of the weights of the LLM. For example, the attention weights and the FCN weights.



LoRA: Low Rank Adaption of LLMs

Step-1: Freeze most of the weights of the LLM. For example, the attention weights and the FCN weights.

Step-2: introduce 2 rank decomposition matrices



LoRA: Low Rank Adaption of LLMs

Step-1: Freeze most of the weights of the LLM. For example, the attention weights and the FCN weights.

Step-2: introduce 2 rank decomposition matrices

Step-3: Train the weights of the smaller matrices

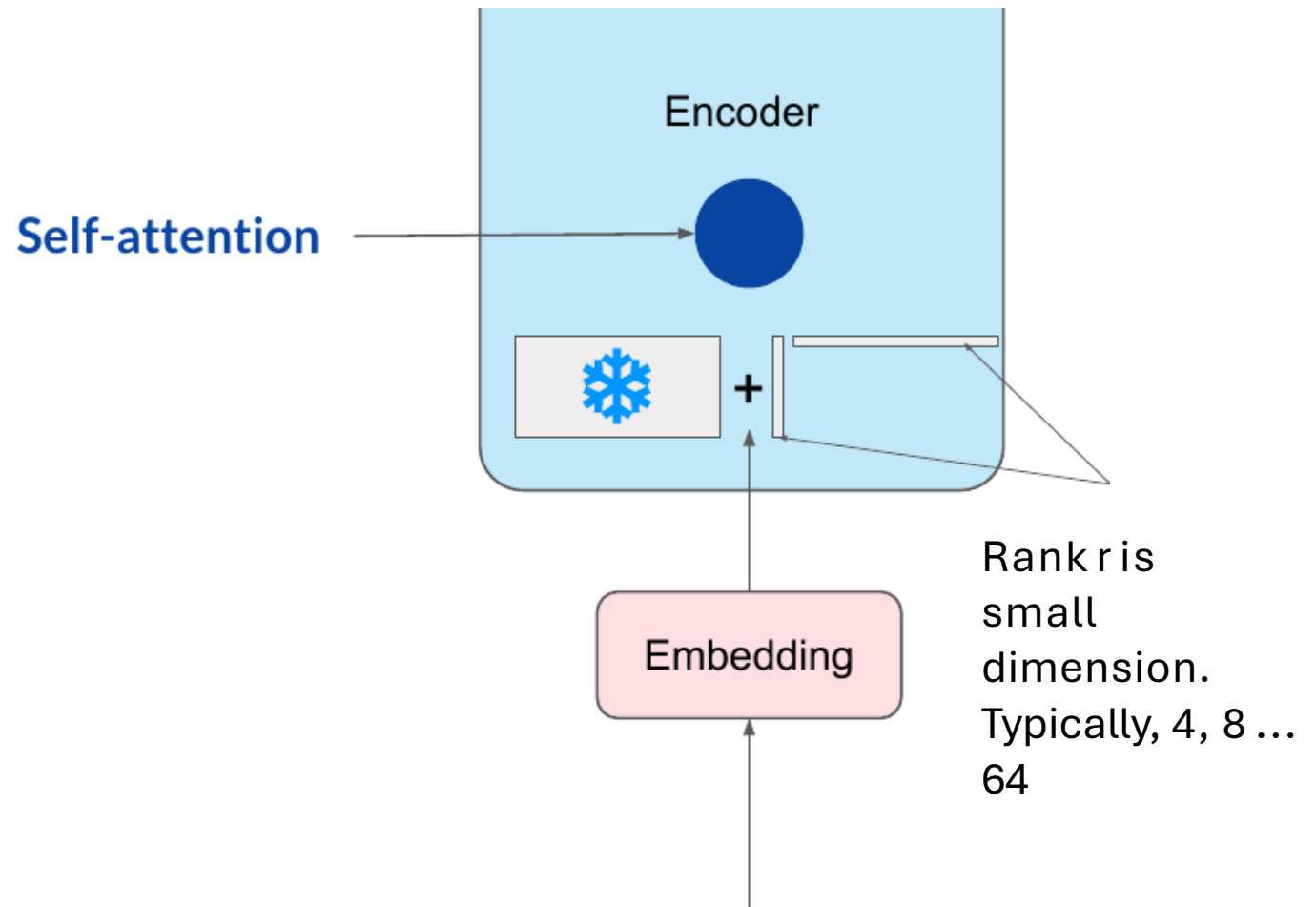
Steps to update model for inference

1. Matrix multiply the low rank matrices

$$\begin{array}{|c|} \hline B \\ \hline \end{array} * \begin{array}{|c|} \hline A \\ \hline \end{array} = \begin{array}{|c|} \hline B \times A \\ \hline \end{array}$$

2. Add to original weights

$$\begin{array}{|c|} \hline \text{Snowflake} \\ \hline \end{array} + \begin{array}{|c|} \hline B \times A \\ \hline \end{array}$$



LoRA: Low Rank Adaption of LLMs

Step-1: Freeze most of the weights of the LLM. For example, the attention weights and the FCN weights.

Step-2: introduce 2 rank matrices decomposition.

Step-3: Train the weights of the smaller matrices

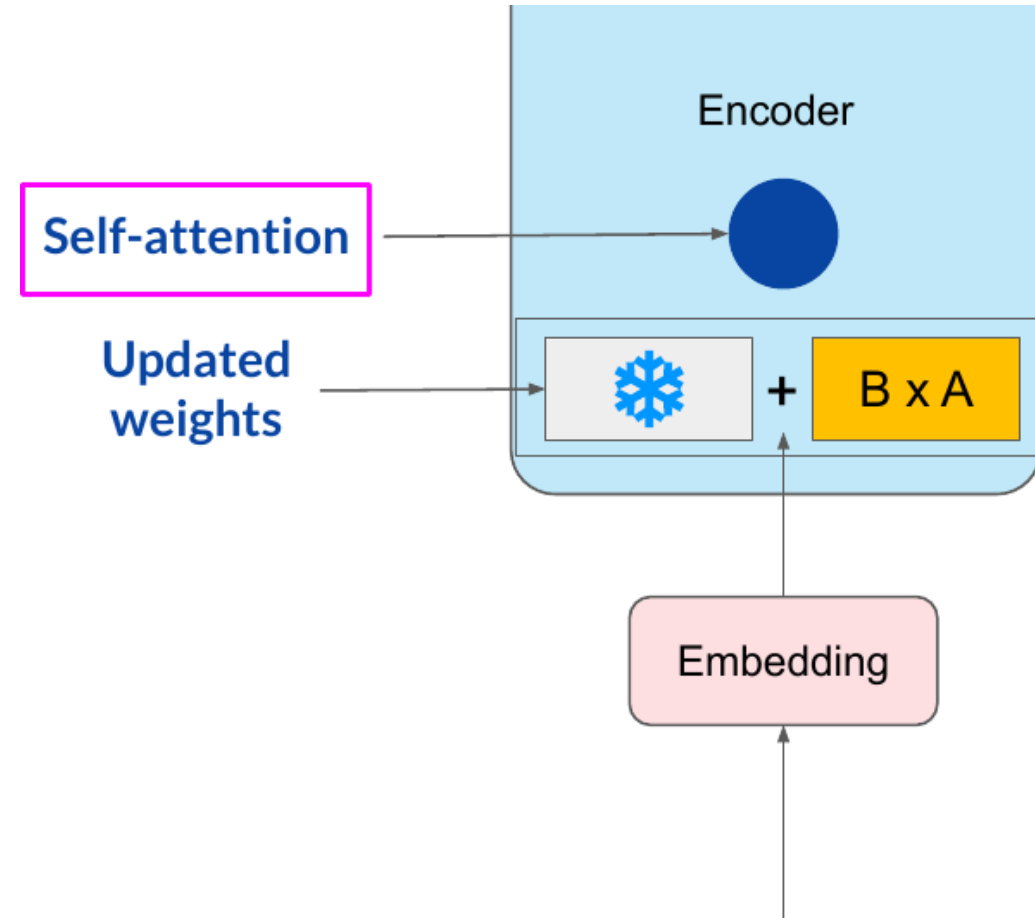
Steps to update model for inference

1. Matrix multiply the low rank matrices

$$B * A = B \times A$$

2. Add to original weights

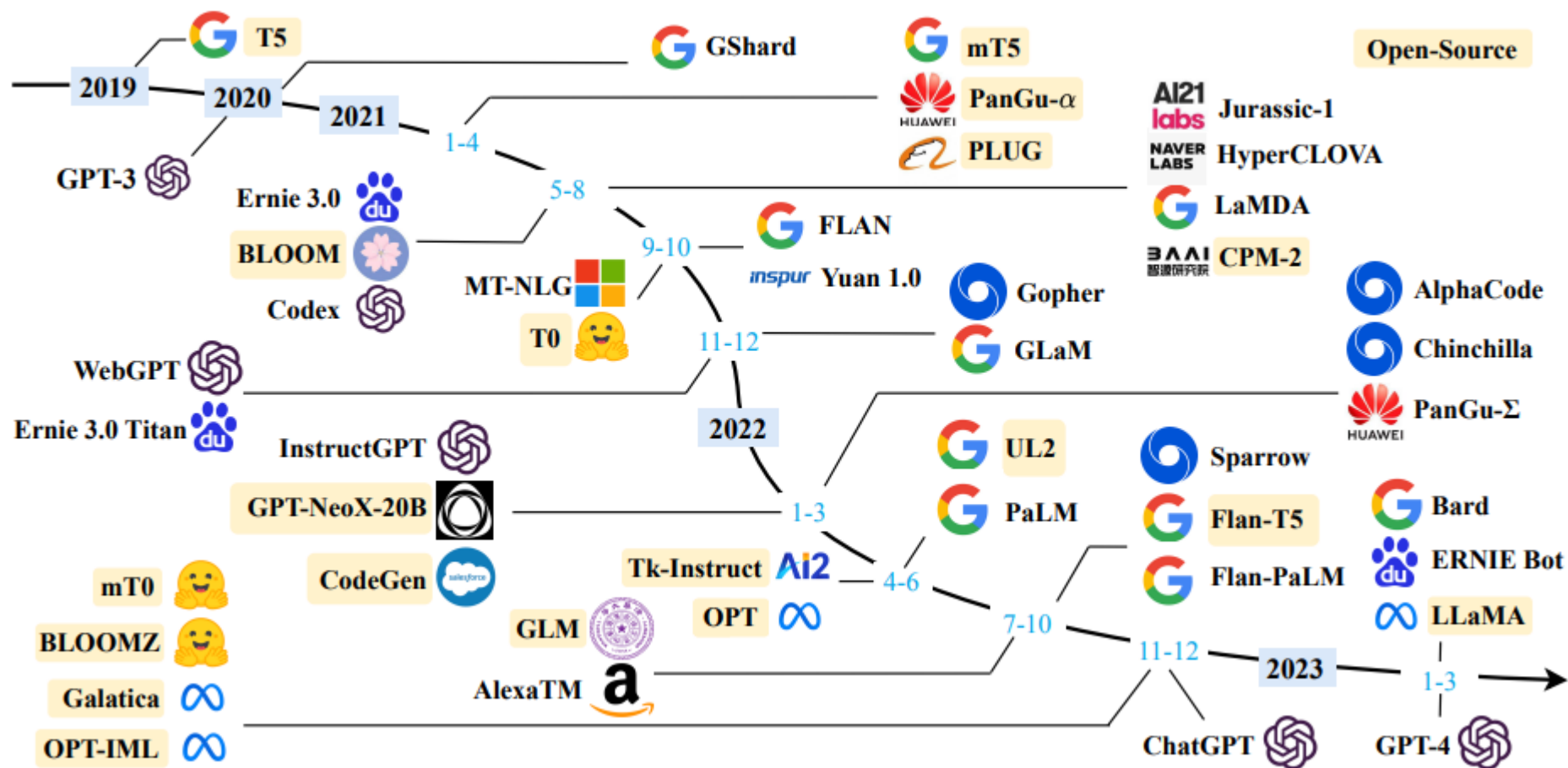
$$\text{Snowflake} + B \times A$$



Lora: Parameter Reduction Example

- Query matrix from BERT second encoder layer has dimensions $d \times k = 768 \times 768$
- So, $768 \times 768 = 5,89,824$ trainable parameters
- In LoRA with rank $r = 32$:
- A has dimensions $r \times k = 32 \times 768 = 9216$ parameters
- B has dimension $d \times r = 768 \times 32 = 9216$ trainable parameters
- 92% reduction in parameters to train!

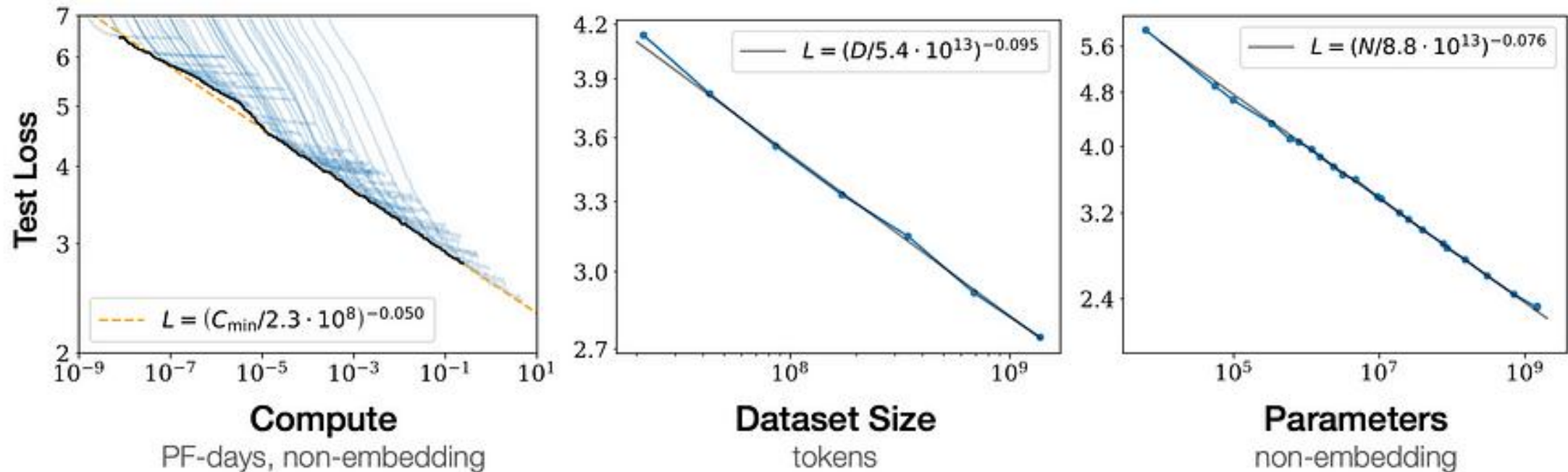
Sample Fine-Tuned (Open Sourced) Models



<https://sungkim11.medium.com/list-of-open-sourced-fine-tuned-large-language-models-llm-8d95a2e0dc76>

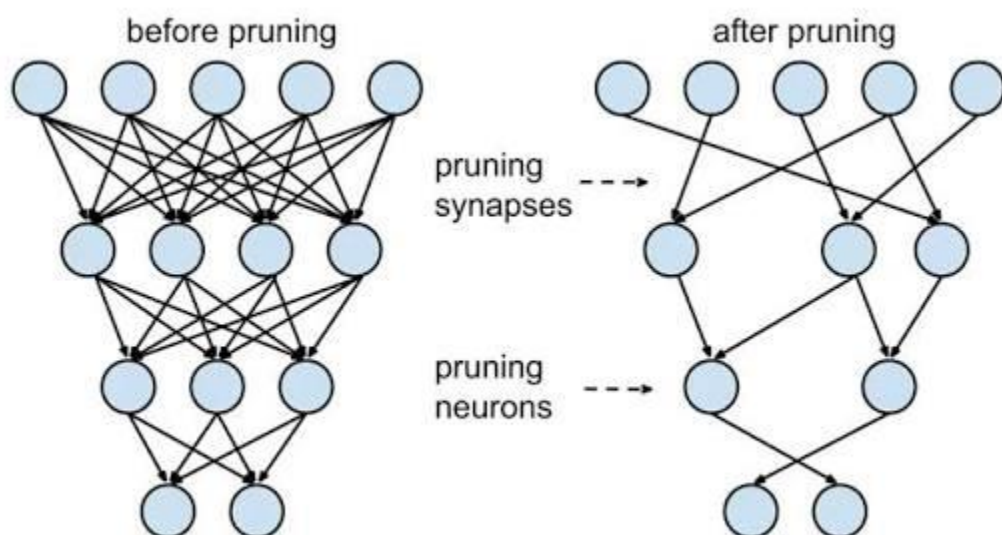
Neural Scaling

Neural Scaling Laws



<https://www.youtube.com/watch?v=5eqRuVp65eY>

Pruning LLMs



- LLM pruning reduces the size & compute cost by removing redundant or less important parameters - weights, neurons, or layers.
- It boosts inference speed, lowers memory consumption, and makes models more efficient for deployment.
- Structured pruning removes structural components (neurons, channels, layers). Useful for edge deployment.
- Unstructured pruning removes individual, non-essential weights, creating sparse matrices. Often requires specialized hardware for acceleration.
- Depth pruning removes complete transformer layers, often at the middle or end, while width pruning narrows the hidden layers.

Quantization



Model Size Reduction

- Most LLMs have billions of parameters → slow, memory-hungry.
- Inference cost is high, especially for cloud and edge deployment.
- Memory constraints limit fine-tuning on smaller GPUs.
- Latency can be a bottleneck for real-time applications.
- Solution: Compress the model without sacrificing much performance.

Smaller LLMs: Quantization

Quantization (Float $\rightarrow$ Int):

$$q = \text{round} \left(\frac{x}{\text{scale}} \right) + \text{zero_point}$$

Scale:

$$\text{scale} = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}} = \frac{x_{\max} - x_{\min}}{255}$$

Dequantization (Int $\rightarrow$ Float):

$$x \approx (q - \text{zero_point}) \times \text{scale}$$

Zero point:

$$\text{zero_point} = \text{round} \left(-\frac{x_{\min}}{\text{scale}} \right)$$

You may then clip or clamp `zero_point` into the valid quantization range.

Term

x

q

scale

zero_point

Meaning

Original float value

Quantized integer

Floating-point step size between quantized levels

Integer that maps float zero to a quantized value (usually in int range)

Quantization

Quantization helps reduce the memory footprint of a model by storing numbers in 8-bit or 4-bit format instead of 32-bit floats.

For example, a 7B parameter model at FP32 takes ~28 GB; INT4 brings it down to ~3.5–4 GB. The tradeoff is slightly lower accuracy, but often acceptable.



Model size vs Quantization

	16-bit memory	8-bit memory	4-bit memory
LLaMA-7B	14 GB	7 GB	3.5 GB
LLaMA-13B	26 GB	13 GB	6.5 GB
LLaMA-30B	60 GB	30 GB	15 GB
LLaMA-70B	140 GB	70 GB	35 GB
GP3-175B	350 GB	175 GB	87.5 GB

Is Quantization a no-brainer, then?

- At the moment, YES!
- 4x-8x reduction in RAM
- Certain implementations using integer math can be faster by 2-3x on most accelerators
- Use of powerful models on EDGE devices possible

QLoRA = Quantized Model + LoRA Adapters

- QLoRA uses **4-bit quantized weights** for the base LLM → drastically reduces memory.
- Adds **LoRA adapters** for efficient fine-tuning → no need to update full weights.
- LoRA adapters are trained in **full precision**; only base model is quantized.
- Enables training of models like **LLaMA-7B on a single 24GB GPU**.
- Matches or exceeds full fine-tuning performance — at a fraction of cost and resources.

QLoRA: Quantized LoRA

- Introduces 4-bit NormalFloat (nf4) data type for 4-bit quantization
- Supports double-quantization to reduce memory ~ 0.4 bits per parameter
- LoRA adapters at every layer - not just attention layers.

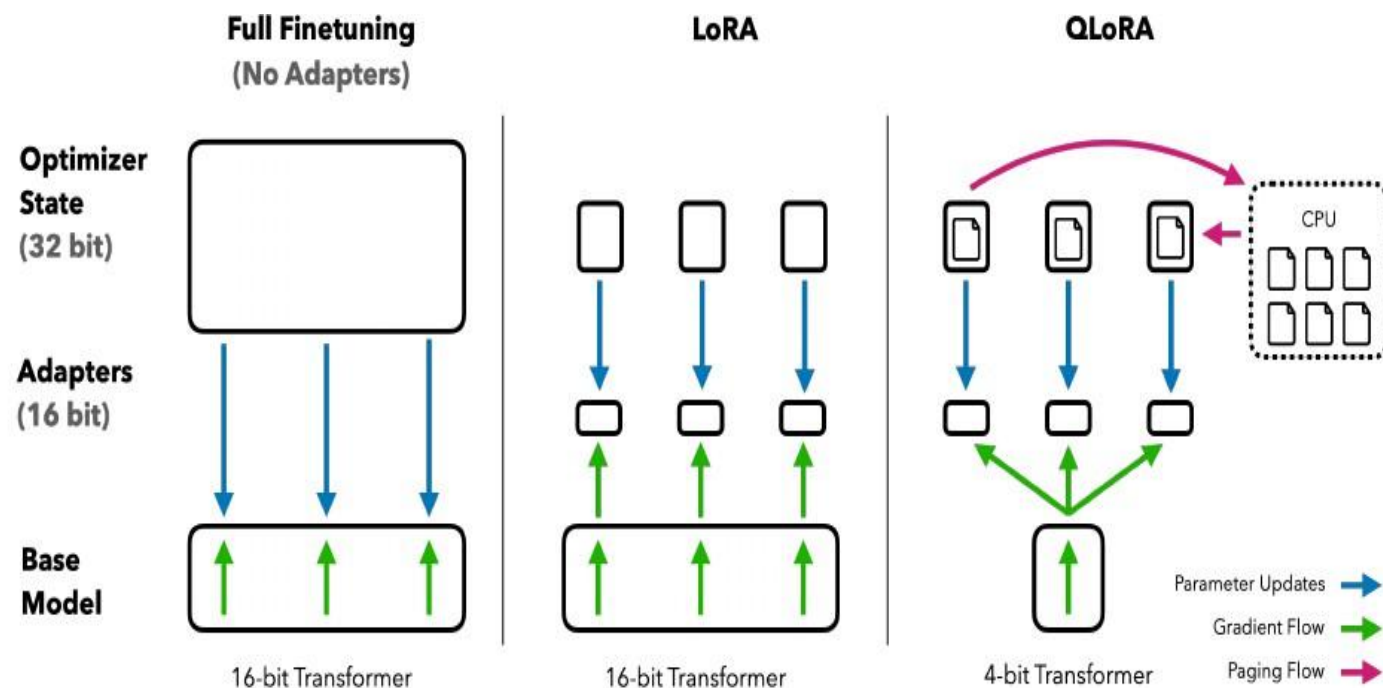


Figure 1: Different finetuning methods and their memory requirements. QLoRA improves over LoRA by quantizing the transformer model to 4-bit precision and using paged optimizers to handle memory spikes.

Distillation



Model Distillation

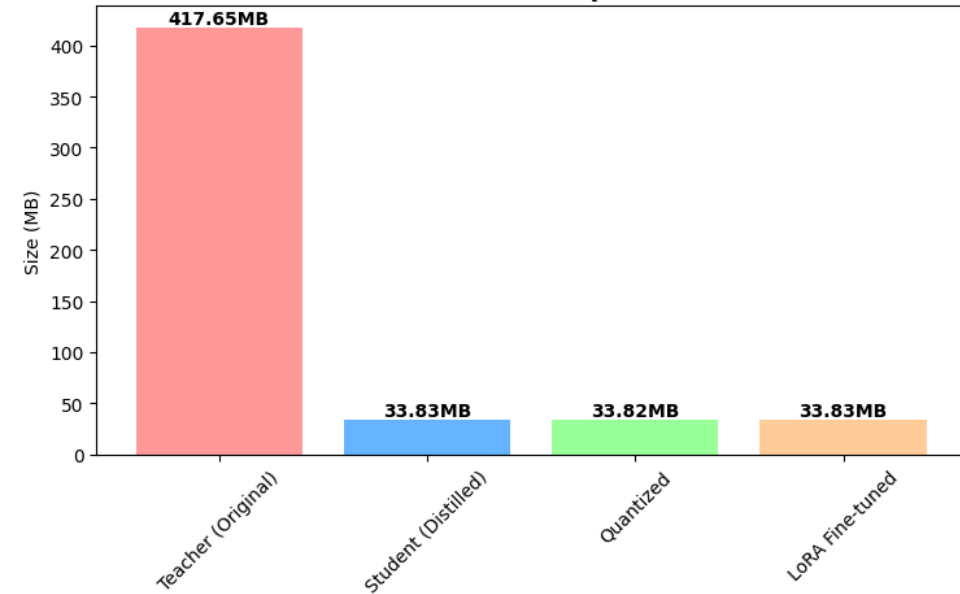
- A “**student model**” learns to mimic the outputs of a larger “**teacher model**”.
- Transfers knowledge from large models to smaller ones — fewer parameters, faster inference.
- Often used to compress models for real-time or mobile deployment.
- The student model learns from both: Ground truth labels; Teacher’s logits or soft outputs (which carry richer info)
- Can preserve accuracy surprisingly well.

Sample List of Distilled Models 2024-26

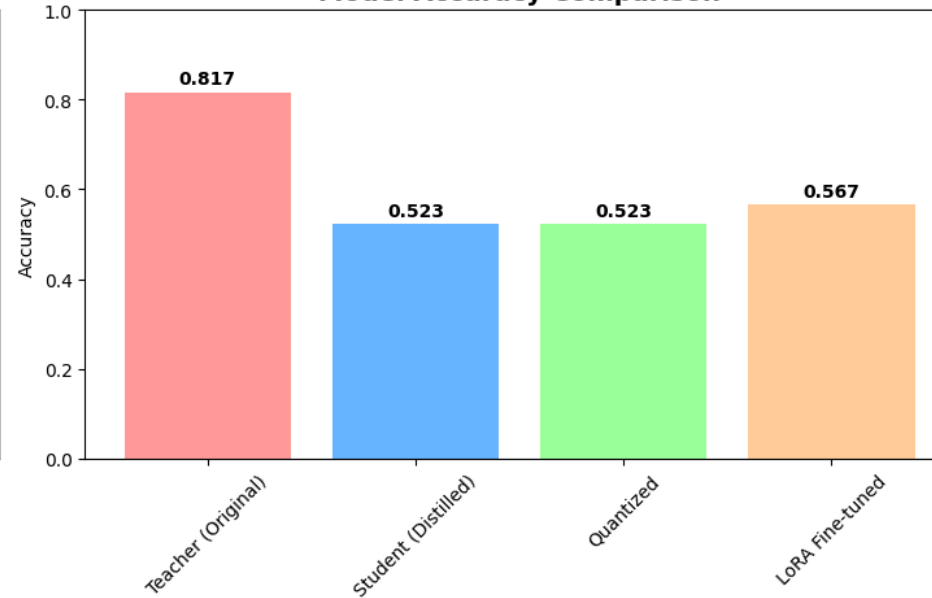
- DeepSeek-R1-Distill-Qwen-1.5B/7B/14B/32B
 - DeepSeek-R1-Distill-Llama-8B/70B
 - Phi-4-mini (3.8B)
 - Phi-3-mini (3.8B)
 - Gemma-2-9B-IT
 - Gemma-2-27B-IT
 - LaMini-LM
 - Zephyr-7B- α and β
 - Orca 2 (7B/13B)
 - DistilBERT and derivatives (e.g., DistilGPT2)
 - Tiny Aya (3.35B)
 - Marcoroni-70B
- 30% lesser parameters
 - 50% faster
 - Retain ~ 95% accuracy

Distilled Model Performance

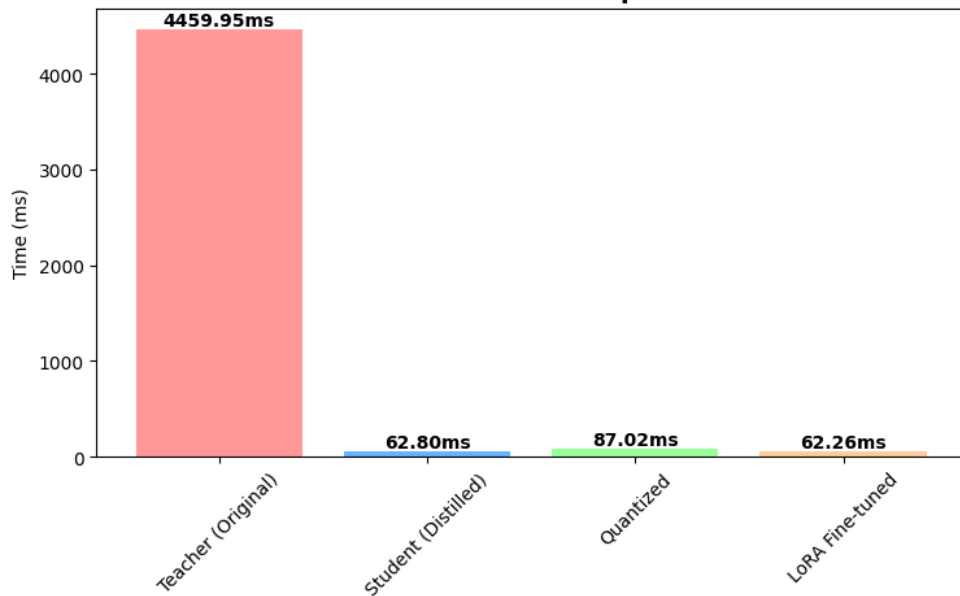
Model Size Comparison



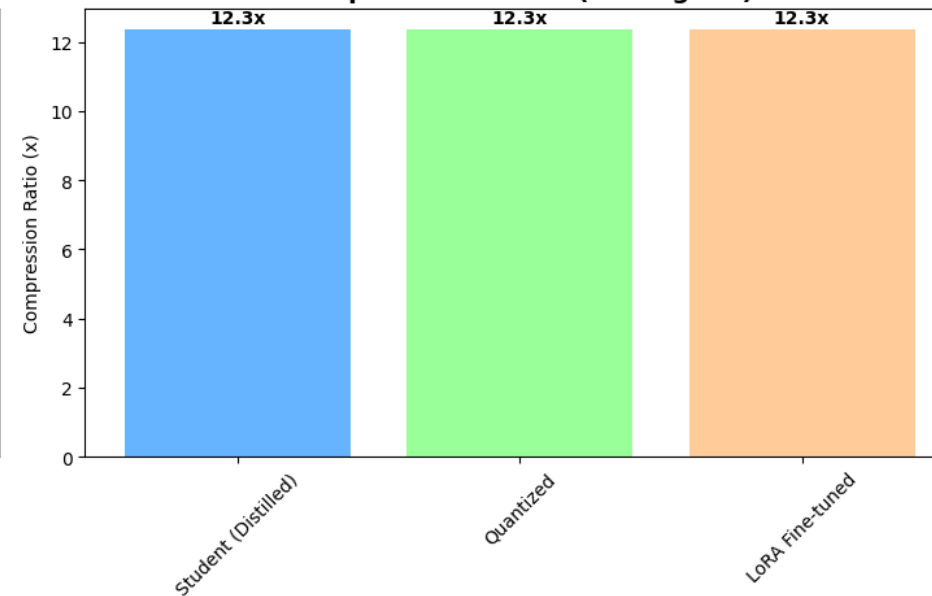
Model Accuracy Comparison



Inference Time Comparison



Compression Ratios (vs Original)



DistilBERT (66M) + 8-bit quantization → ~15MB model

Used in **mobile NLP apps** (e.g., **Snips Voice Assistant**) for fast, offline inference with minimal accuracy loss.

Decision Matrix

When to use what

Distillation vs Quantization

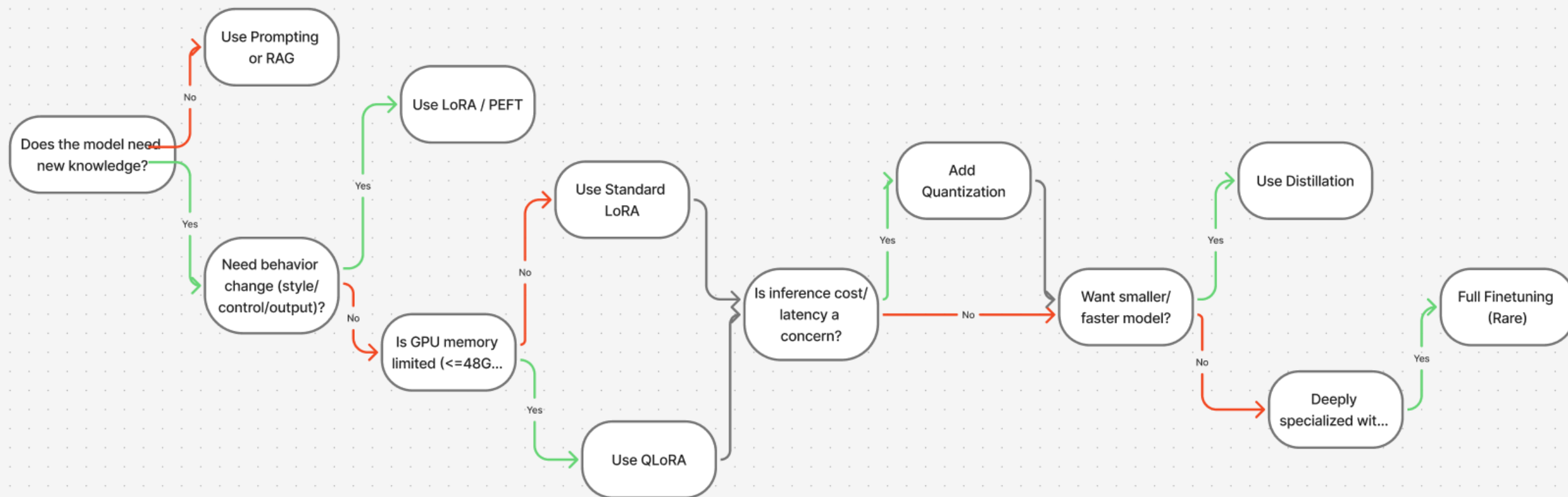
Feature	Distillation	Quantization
What it does	Trains a small model to mimic a large model	Reduces weight precision (e.g., FP32 → INT4)
Model Architecture	Student model is separate from teacher	Model architecture remains the same
Training Needed	✅ Yes – Student needs to be trained	❌ No (Post-training quantization is common)
Accuracy Impact	Moderate loss (can be minimized)	Slight accuracy drop depending on bits used
Use Case	Create fast, lightweight inference models	Deploy large models on small hardware
Combines With	PEFT, task-specific tuning	LoRA (→ QLoRA), on-device models

Choosing the right model optimization technique

Scenario / Need	Best Method	Why
Want lowest latency + smallest memory footprint	Quantization (4-bit / 8-bit)	Massive RAM reduction, deploy on low-cost GPUs/edge devices
Want small model that behaves like a big model	Distillation	Student captures teacher behavior; fast inference
Want to add <i>new behavior</i> , style, or output patterns	PEFT / LoRA	Efficient weight updates, minimal compute, preserves base LLM
Want to fine-tune <i>quantized model</i> on limited GPU (e.g., 24GB)	QLoRA	Full finetuning performance on 4-bit model
Need domain-context answers, but not new model behavior	RAG (next module)	External knowledge → no model changes
Need full weights updated for highly specialized tasks	Full Finetuning	Maximum flexibility, but highest compute/storage cost

Most enterprise tasks can be solved with Quantization + LoRA / QLoRA. Full finetuning is rarely needed today

Practical Decision Flow for Enterprises



Thank You

We appreciate your time and look forward to your feedback on the session.

Contact us: TekFrameworks Consulting | info@tekframeworks.com | www.tekframeworks.com

Confidentiality Notice: This proposal is proprietary to **TekFrameworks Consulting Pvt. Ltd.** and is intended solely for the Client's evaluation. It may not be shared with third parties. The contents are non-binding; any engagement will be governed by a separate written agreement. For Client Use Only | TekFrameworks Consulting Pvt. Ltd. | © 2025 TekFrameworks